

Statistical Programming Languages

Graphics

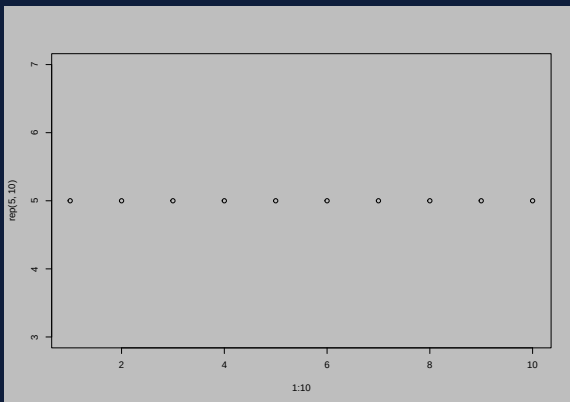
Katarzyna Reluga¹

¹Acknowledgements: Thanks to Manuel Pfeuffer, Maarten Jung, and Tobias Wistub for sharing notes from earlier editions.

Basic plot examples

You may pass two vectors of the same length to `plot()`

```
# setting the background color for captions' visibility  
par(bg = "grey")  
plot(x = 1:10, y = rep(5, 10))
```



Basic plot examples

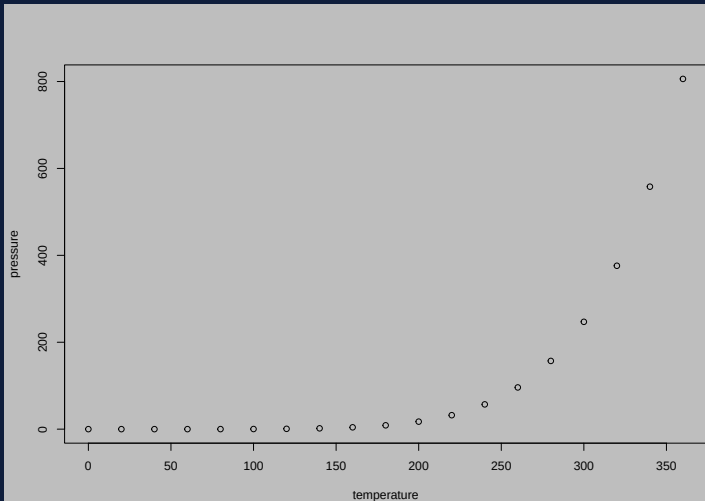
You may also pass a simple two-column data frame or matrix to `plot()`

```
# Load pressure dataset
data(pressure)
head(pressure)

##      temperature pressure
## 1             0  0.0002
## 2            20  0.0012
## 3            40  0.0060
## 4            60  0.0300
## 5            80  0.0900
## 6           100  0.2700
```

Basic plot examples

```
# Create a simple scatter plot  
par(bg = "grey")  
plot(pressure)
```



R graphics devices

- ▶ Save a plot object using a **graphics device**
- ▶ Standard devices are pdf, png, jpeg, ...
- ▶ `dev.off()` closes devices sequentially

```
pdf(file = "Rplot.pdf", width = 12, height = 12)  
plot(pressure)  
dev.off()
```

Labels and axes

- ▶ R relies on graphical parameters for styling the plot
- ▶ Parameters may be changed for a plot by passing them to the plot function
- ▶ The following parameters refer to labels and axes
 - ▶ axes labels: `xlab`, `ylab`
 - ▶ size of labels: `cex.lab`
 - ▶ axes range: `xlim`, `ylim`
- ▶ Use `?par` for help on graphical parameters

Symbols, colors, sizes for points, etc.

Point symbols, colors, and sizes can be changed using the following parameters

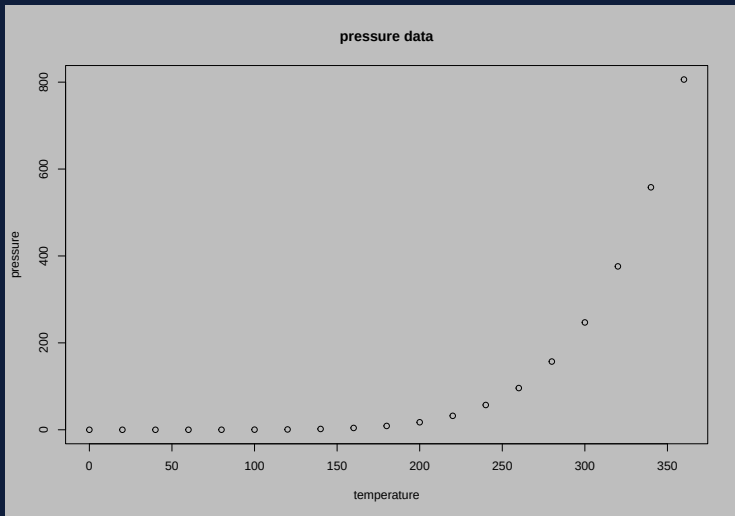
- ▶ point symbol: `pch (0:25)`
- ▶ `col`: color (try `colors()`)
- ▶ `cex`: size factor

Titles and lines

- ▶ `main`, `xlab`, `ylab`: as the parameters of the `plot`
- ▶ Alternative: add main title, subtitle, ... to already existing plot with separate function `title()`
- ▶ `type`: “p” for points, “l” for lines, “b” for both, “h” for histogram, etc.

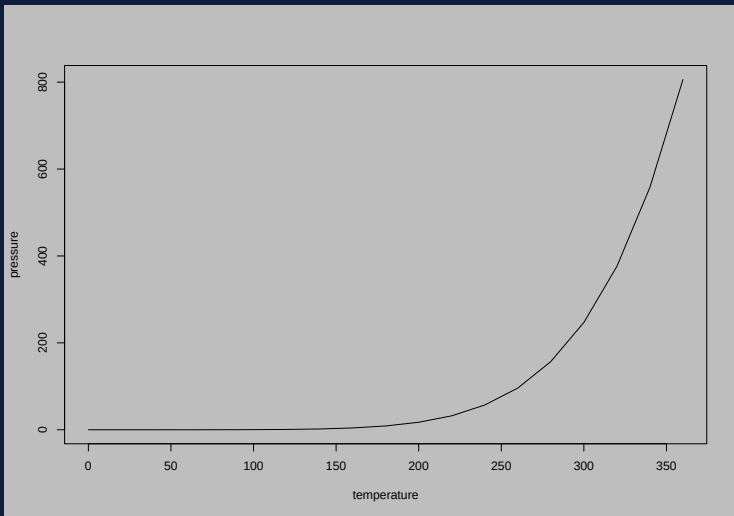
Plot examples

```
par(bg = "grey")  
plot(pressure, type = "p", main = "pressure data")
```



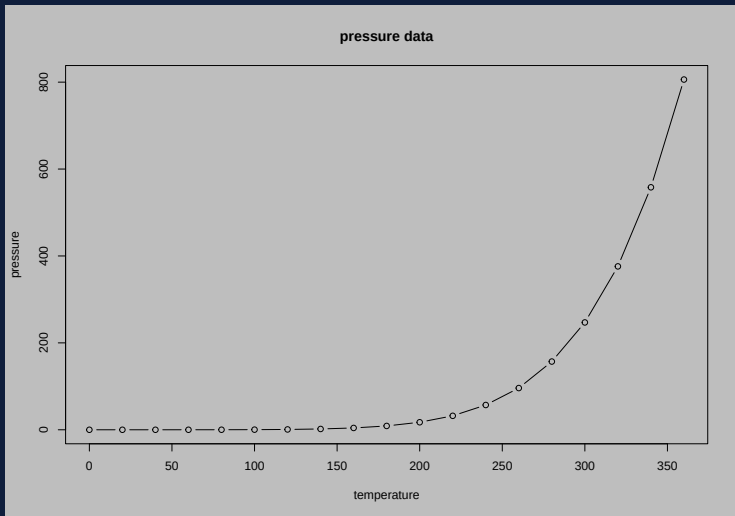
Plot examples

```
par(bg = "grey")  
plot(pressure, type = "l")
```



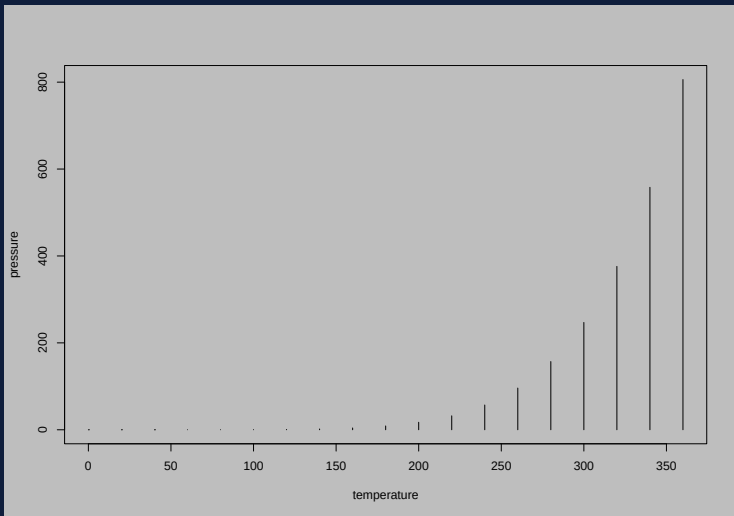
Plot examples

```
par(bg = "grey")  
plot(pressure, type = "b", main = "pressure data")
```



Plot examples

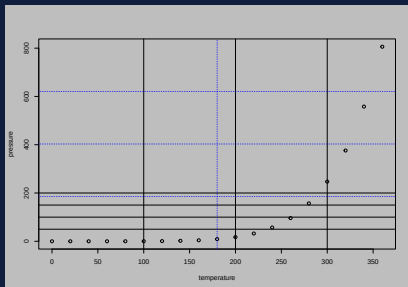
```
par(bg = "grey")  
plot(pressure, type = "h")
```



Grid and legend

- ▶ horizontal and vertical lines: `abline(h = seq(.,.,.))` or `abline(v = seq(.,.,.))`
- ▶ alternative: `grid`
- ▶ add a legend to a plot using the function `legend()`

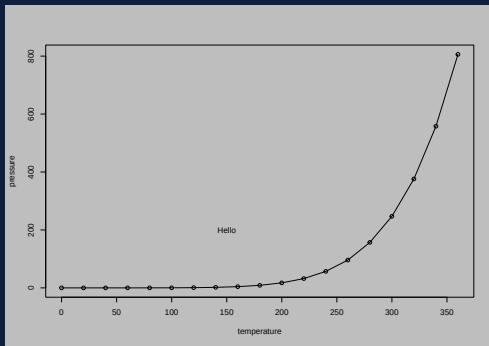
```
par(bg = "grey")  
plot(pressure, type = "p")  
abline(h = seq(50, 200, 50))  
abline(v = seq(100, 600, 100))  
grid(nx = 2, ny = 4, col = "blue") #nx, ny num. of cells
```



Adding text

We can add text to a plot at any position

```
data(pressure)
par(bg = "grey")
plot(pressure)
lines(pressure)
text(150, 200, label = "Hello")
```

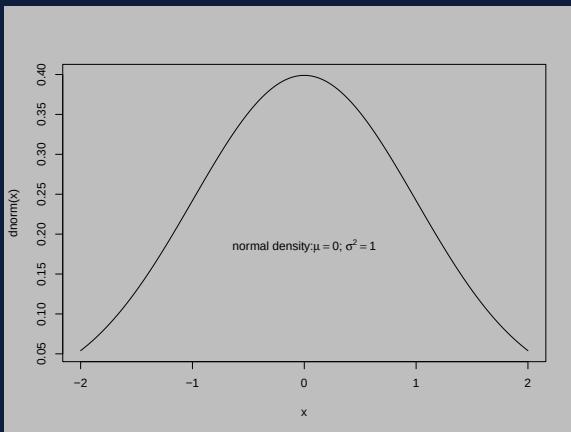


Adding math to plots

- ▶ We may include symbols in text and labels of plots
- ▶ Symbols are available as expressions
- ▶ Best use the `bquote()` function

```
# Plot a vector x
x <- seq(-2, 2, length.out = 200)
par(bg = "grey")
plot(x, dnorm(x), type = "l")
# Add text to the plot
mu <- 0
variance <- 1
# pos = position of the label relative to (x, y)
# pos = 1 puts the label just below the point (0, 0.2)
text(0, 0.2 , pos = 1,
     label = bquote(paste("normal density:",
                           mu == .(mu), "; ",
                           sigma^2 == .(variance))))
```

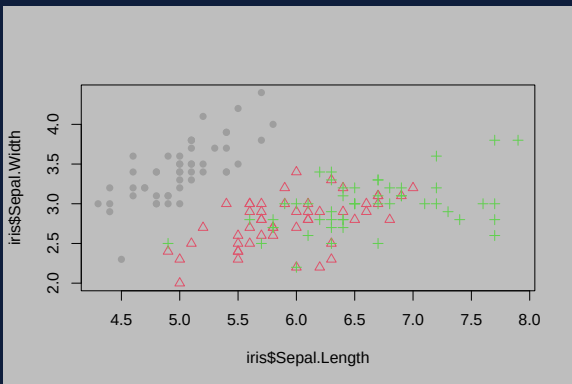
Adding math to plots



Distinguish variables inside a plot

```
par(bg = "grey")
data(iris) # load iris data
# Create vectors assigning point styles and
# colors to different levels of the factor variable
pch.vec <- c(16, 2, 3)[iris$Species]
col.vec <- c(16, 2, 3)[iris$Species]
plot(iris$Sepal.Length, iris$Sepal.Width,
      col = col.vec, pch = pch.vec)
```

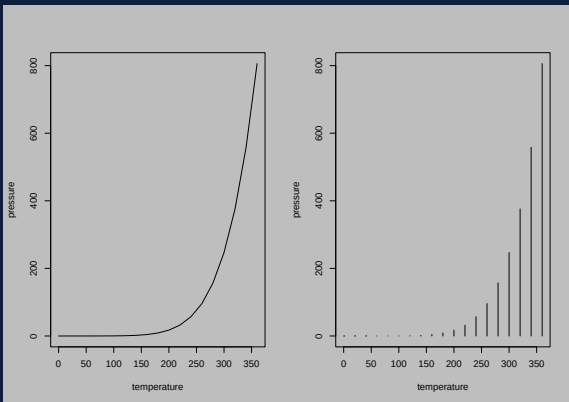
Distinguish variables inside a plot



Side by side plots

- ▶ Arranging similar plots in a grid is useful
- ▶ Global parameter `mfrow` helps achieving this

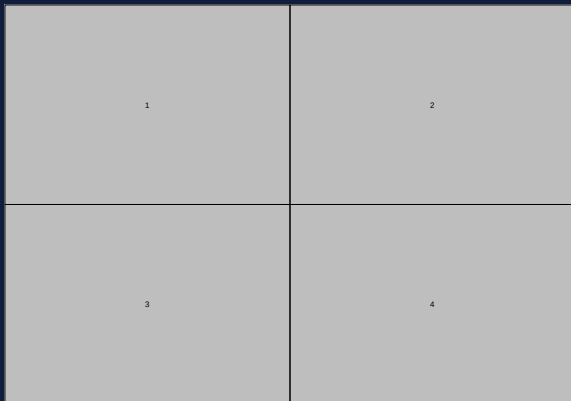
```
par(mfrow = c(1, 2), bg = "grey") # two-column plot  
plot(pressure, type = "l")  
plot(pressure, type = "h")
```



Side by side plots

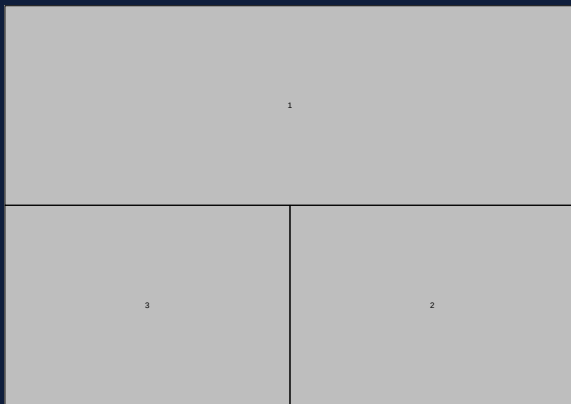
- ▶ `mfrow` is nice
- ▶ Alternative: `layout()` function

```
par(bg = "grey")  
layout(matrix(1:4, 2, 2, byrow = TRUE))  
layout.show(4)
```



Side by side plots

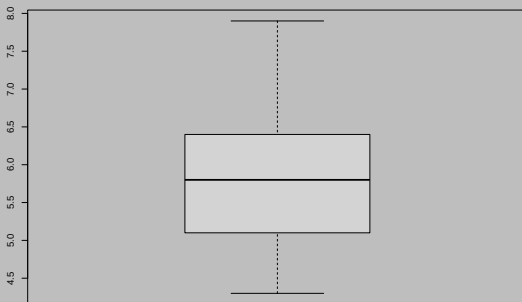
```
par(bg = "grey")  
layout(matrix(c(1, 1, 3, 2), 2, 2, byrow = TRUE))  
layout.show(3)
```



Boxplot

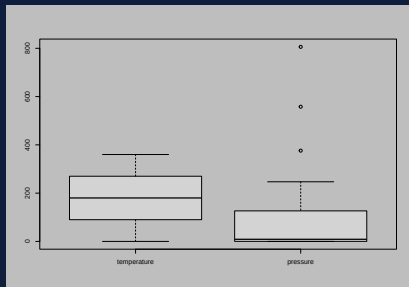
Boxplots are useful for visualising the distribution of data (range, variability, outliers, etc.)

```
# boxplot for iris variable  
par(bg = "grey")# median, Q1 & Q3 quartiles, whiskers  
# extend to points within 1.5 from Q1/Q3, outliers  
boxplot(iris$Sepal.Length)
```



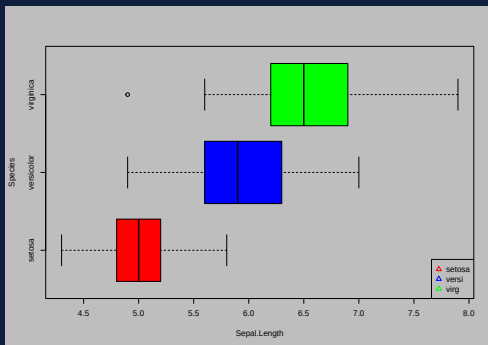
Boxplot

```
# boxplot for pressure and temperature  
par(bg = "grey")  
boxplot(pressure, col = "lightgray")
```



Boxplot separated by factors

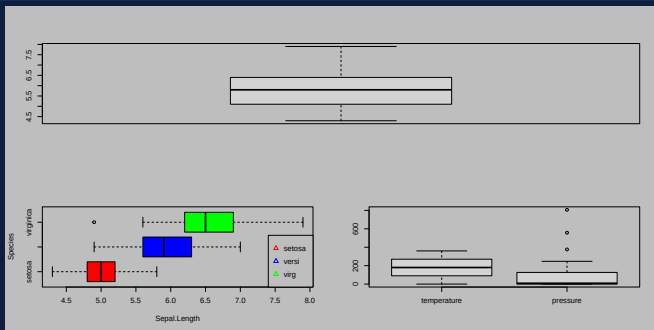
```
par(bg = "grey")  
boxplot(Sepal.Length~Species, data = iris,  
        horizontal = TRUE,  
        col = c("red", "blue", "green"))  
legend("bottomright", pch = c(2, 2, 2),  
      legend = c("setosa", "versi", "virg"),  
      col = c("red", "blue", "green"))
```



Example

- All three plots together with layout of our choice

```
#layout(matrix(c(1, 1, 3, 2), 2, 2, byrow = TRUE))  
#layout.show(3)
```



Histograms

Histograms are useful for visualising the distribution of data (range, variability, outliers, etc.) as well

```
# Histogram with frequencies  
par(bg = "grey")  
hist(iris$Sepal.Length, col = "lightblue")
```



Histograms

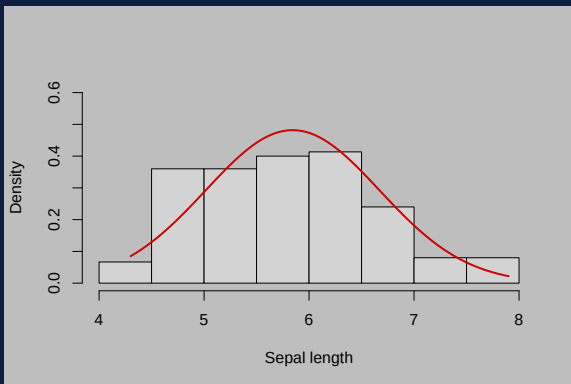
```
# Histogram with density  
par(bg = "grey")  
hist(iris$Sepal.Length, freq = FALSE)
```



Histogram with dnorm

```
# Histogram with density
par(bg = "grey")
hist(iris$Sepal.Length, freq = FALSE,
      ylim = c(0, 0.6), xlab = 'Sepal length',
      main = NULL)
# Create a grid of points to be passed to lines()
g <- seq(min(iris$Sepal.Length),
          max(iris$Sepal.Length), length.out = 100)
# Compute the normal density on g
dens.norm <- dnorm(g, mean(iris$Sepal.Length),
                    sd(iris$Sepal.Length))
# Add density to plot as line
lines(g, dens.norm, col = "red3", lwd = 2)
```

Histogram with dnorm



Nicer plots with package ggplot2

- ▶ ggplot2 is based on the book “Grammar of Graphics” by Leland Wilkinson (2005)
- ▶ The package uses a highlevel graphics syntax that allows to stack different levels of data visualizations
- ▶ Often better to create nice looking plots
- ▶ There are many places to get help with ggplot2: RStudio Community, Stack Overflow, chat boxes, etc

Grammar of Graphics

- ▶ All `ggplot2` plots start with a call to `ggplot()`
- ▶ We first set the default data and aesthetic mappings with `aes(...)`
- ▶ Then we add layers (geoms), scales, and facets using `+`:
 - ▶ layers: `geom_point()`, `geom_abline()`, `geom_histogram()`, `geom_bar()` (geom. layers); `stat_ecdf()` and `stat_summary()` (stats layers), etc
 - ▶ scales: `xlab()`, `ylab()`, `ggtitle()`, `xlim()`, `ylim()`, etc
 - ▶ faceting: `theme()`

Basic scatter plot

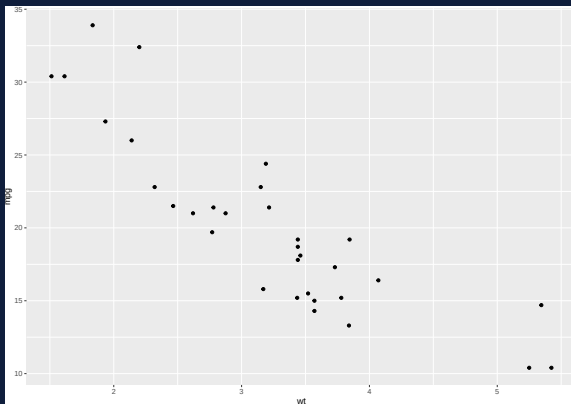
```
# install ggplot2 if necessary
# install.packages("ggplot2")
library(ggplot2)
data(mtcars)
head(mtcars, 2)
```

##		mpg	cyl	disp	hp	drat	wt	qsec
##	Mazda RX4	21	6	160	110	3.9	2.620	16.46
##	Mazda RX4 Wag	21	6	160	110	3.9	2.875	17.02

##		vs	am	gear	carb
##	Mazda RX4	0	1	4	4
##	Mazda RX4 Wag	0	1	4	4

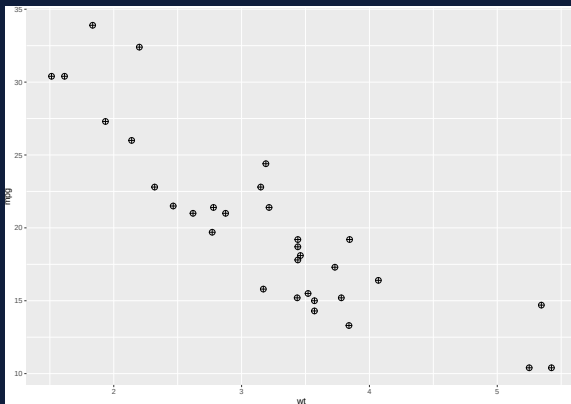
Basic scatter plot

```
# Basic scatter plot  
ggplot(data = mtcars, mapping = aes(x = wt, y = mpg)) +  
  geom_point()
```



Basic scatter plot

```
# Change the point size, and shape  
ggplot(data = mtcars, aes(x = wt, y = mpg)) +  
  geom_point(size = 3, shape = 10)
```

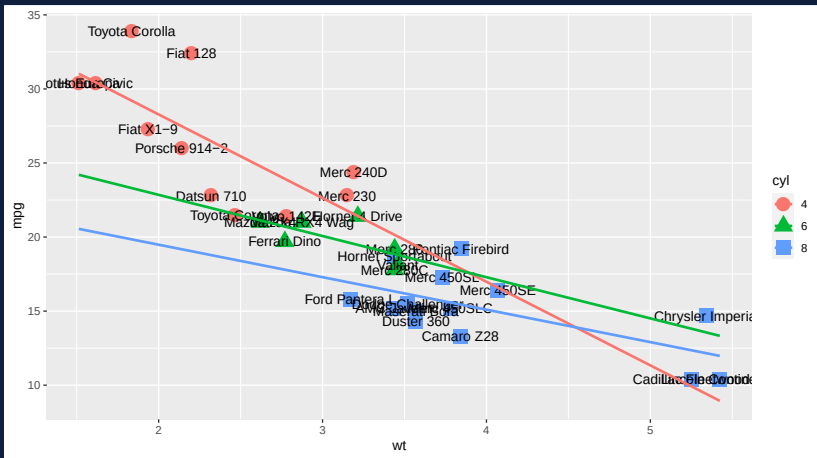


Fancier scatter plot

```
# Change the class of cyl from "numeric" to "factor"
class(mtcars$cyl); unique(mtcars$cyl)
## [1] "numeric"
## [1] 6 4 8
mtcars$cyl <- as.factor(mtcars$cyl)

ggplot(data = mtcars, aes(x = wt, y = mpg)) +
  # points by cyl
  geom_point(size = 5, aes(color = cyl, shape = cyl)) +
  # plotting labels
  geom_text(aes(label = rownames(mtcars))) +
  # fit a lm by cyl, aes(color = cyl) --> 3 lines
  geom_smooth(aes(color = cyl), method = lm,
              se = FALSE, fullrange = TRUE)
## `geom_smooth()` using formula = 'y ~ x'
```

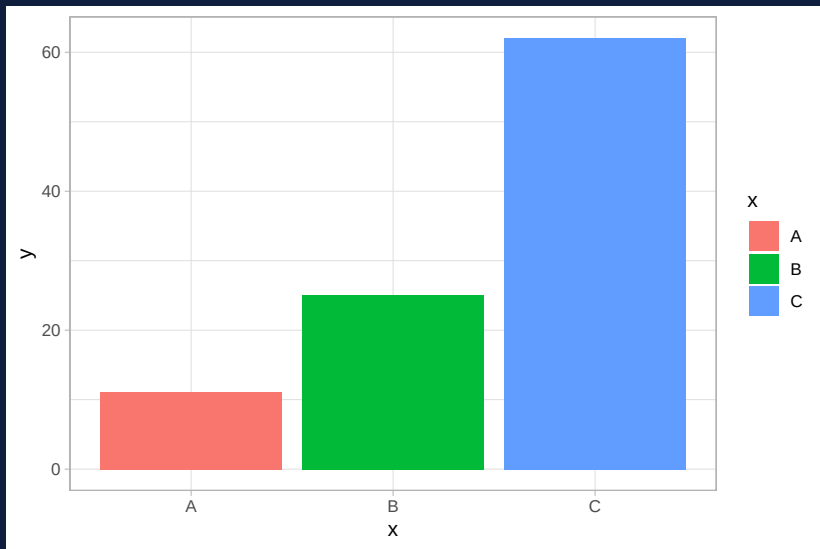
Fancier scatter plot



Bar plot

```
mydata <- data.frame(x = rep(c("A", "B", "C"), 2),  
                     y = c(6.8, 15, 33, 4.2, 10, 29))  
  
ggplot(mydata, aes(x = x, y = y)) +  
  # bar-plot  
  geom_bar(aes(fill = x), stat = "identity") +  
  theme_light()  
# fill matches "A", "B", "C" to different fill colours  
# stat = "identity"; use your y values as-is  
# (no counting or summarising).
```

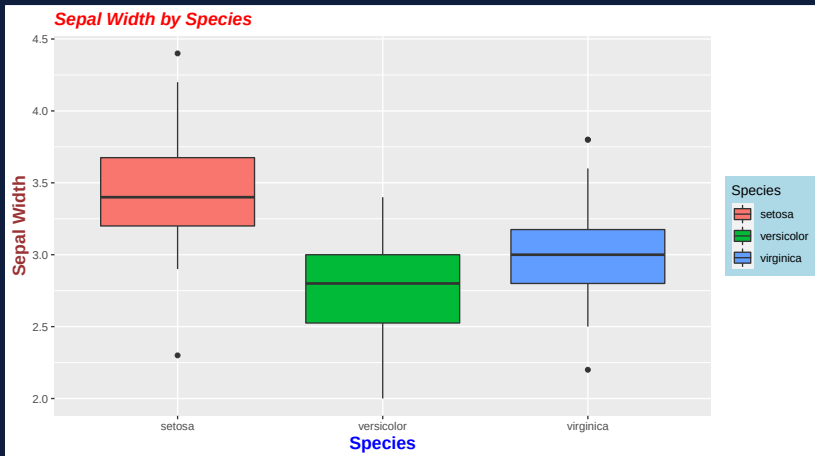
Bar plot



Box plot

```
data(iris)
ggplot(iris, aes(x = Species, y = Sepal.Width,
                 fill = Species)) +
  geom_boxplot() +
  # main and axes titles
  labs(title = "Sepal Width by Species",
       x = "Species", y = "Sepal Width") +
  # titles styling
  theme(plot.title = element_text(color = "red",
                                   size = 14, face = "bold.italic"),
        axis.title.x = element_text(color = "blue",
                                      size = 14, face = "bold"),
        axis.title.y = element_text(color = "#993333",
                                      size = 14, face = "bold"),
        # legend styling
        legend.background =
          element_rect(fill = "lightblue"))
```

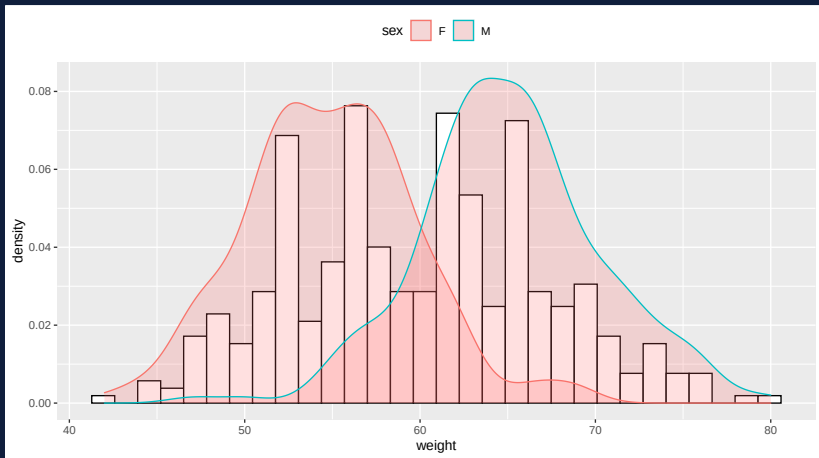
Box plot



Histogram

```
mydata <- data.frame(  
  sex = factor(rep(c("F", "M"), each = 200)),  
  weight = round(c(rnorm(200, mean = 55, sd = 5),  
                  rnorm(200, mean = 65, sd = 5)))  
  
ggplot(mydata, aes(x = weight, color = sex)) +  
  geom_histogram(aes(y = after_stat(density)),  
                 colour = "black", fill = "white") +  
  geom_density(alpha = .2, fill = "#FF6666") +  
  theme(legend.position = "top")  
# alpha: controls transparency, 0 = fully transparent  
# 1 = fully opaque
```

Histogram

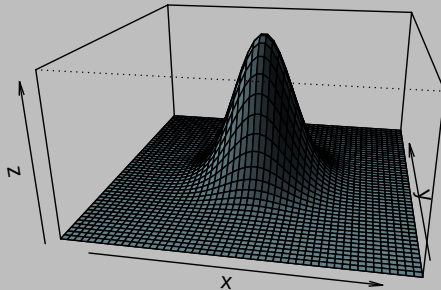


Multivariate plots

Create a snapshot of a 3-D plot (handy for 3-D random variables)

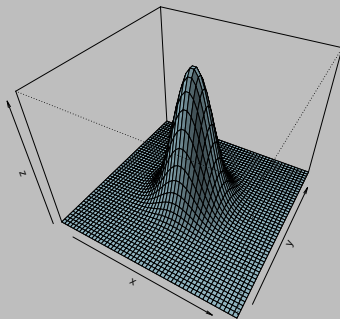
```
# install.packages("mvtnorm")
library(mvtnorm)
# Define a grid of x and y values
x <- y <- seq(-5, 5, length.out = 50)
# Function that computes the multivar. normal density
f <- function(x, y) { dmvnorm(cbind(x, y)) }
# Compute the multi. normal density for all pairs x, y, f
z <- outer(x, y, f)
# Create a 3D plot
par(bg = "grey")
persp(x, y, z, theta = 10, # rotate view around z-axis
      phi = 20, # tilt camera from above toward horizon
      expand = 0.5, # vertical scaling of z
      col = "lightblue", # surface color
      shade = 0.75) # shading strength
```

Multivariate plots



Multivariate plots

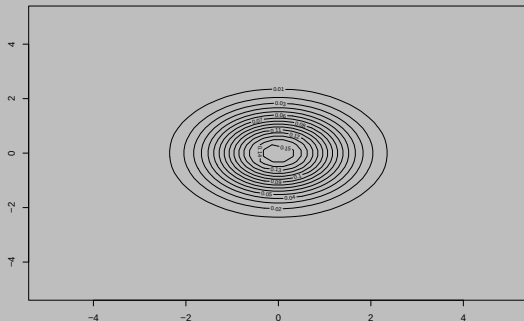
```
# Different viewing angles  
par(bg = "grey")  
persp(x, y, z, theta = 30, phi = 40, expand = 0.8,  
      col = "lightblue", shade = 0.25)
```



Contour plots

- ▶ 2D visualization of a 3D plot
- ▶ Mapping of an object in $\mathbb{R}^3$ to a plane in $\mathbb{R}^2$

```
# contour curves for bivariate density  
par(bg = "grey")  
contour(x, y, z, nlevels = 20)
```



```
par(bg = "grey")  
contour(x, y, z, nlevels = 20, col = rainbow(20))
```

